

Below is a **technical Product Requirements Document (PRD)** for **WeUP**, split cleanly by **subsystem**.

This is written for **engineering, architecture, and product leadership**—not marketing.

Each subsystem includes: **purpose, scope, core requirements, non-goals, and technical notes** so it can be handed directly to teams or broken into epics.

WeUP — Technical PRD

(Subsystem-Oriented)

0. System Overview (Context for All Subsystems)

WeUP is a distributed, invite-based event coordination platform with blockchain-anchored identity, ritualized participation mechanics, spatial discovery, and event-bound commerce.

The system prioritizes:

- Presence over engagement
 - Trust over virality
 - Lineage over follower counts
 - Events as first-class objects
-

1. Identity & Access Subsystem

Purpose

Provide pseudonymous, tiered, lineage-aware identity without relying on traditional social profiles.

Scope

- User identity
- Invite lineage
- Tiered access control

- Wallet binding

Functional Requirements

- Users must have a unique internal identity UUID
- Identity may optionally bind to a blockchain wallet
- Identity must support **tier assignment**
- Identity must track **invite lineage graph**
- Identity must support **event-scoped anonymity**
- Tier upgrades/downgrades must be auditable

Non-Goals

- No real-name enforcement
- No public follower graphs
- No social feed identity mechanics

Technical Notes

- Identity service (authoritative)
 - Graph structure for invite lineage
 - Wallet abstraction layer (chain-agnostic)
 - Zero-knowledge compatible in future phases
-

2. Invite & Tier System

Purpose

Control access, trust propagation, and cultural quality through structured invitations.

Scope

- Invite generation
- Tier inheritance rules
- Invite decay and revocation

Functional Requirements

- Invites must be generated by authorized tiers
- Invites must encode:
 - Issuer

- Recipient
 - Tier granted
 - Expiry rules
- Invites must be revocable
- Tier effects must propagate conditionally (not automatically)

Non-Goals

- No open signups
- No viral invite loops

Technical Notes

- Invite tokens stored off-chain initially
 - Optional on-chain anchoring later
 - Deterministic rule engine for tier logic
-

3. Event Core Subsystem

Purpose

Model real-world gatherings as programmable, contract-aware objects.

Scope

- Event creation
- Event lifecycle
- Participation states

Functional Requirements

- Events must support:
 - Public / private / invite-only visibility
 - Capacity limits
 - Temporal constraints
 - Location masking
- Event lifecycle states:
 - Draft
 - Curated
 - Active
 - Live

- Archived
- Events must emit lifecycle events to other subsystems

Non-Goals

- No infinite event listings
- No algorithmic boosting of events

Technical Notes

- Event service as core domain service
 - Strong consistency required
 - Emits domain events (Kafka/RabbitMQ)
-

4. Ticketing & Attendance Subsystem

Purpose

Gate access and prove participation without commoditizing attendance.

Scope

- Ticket issuance
- Entry verification
- Attendance proof

Functional Requirements

- Tickets must be bound to identity or wallet
- Transferability rules configurable per event
- Tickets must support:
 - QR / NFC / cryptographic proof
- Attendance must produce a verifiable receipt
- Tickets may transform post-event (burn, archive, artifact)

Non-Goals

- No speculative resale markets
- No open ticket marketplaces

Technical Notes

- Abstract ticket interface
 - NFT optional, not mandatory
 - Offline verification support
-

5. Ritual & Attention System

Purpose

Actively reduce phone addiction and encourage presence during events.

Scope

- Ritual definitions
- Compliance tracking
- Incentive logic

Functional Requirements

- Events may define ritual phases
- Rituals may include:
 - Phone-off windows
 - Collective actions
 - Synchronized moments
- Compliance must be opt-in
- Compliance must generate:
 - Access unlocks
 - Reputation effects
 - Post-event privileges

Non-Goals

- No punitive enforcement
- No biometric surveillance

Technical Notes

- Ritual engine separate from event engine
 - Time-boxed state machines
 - Privacy-preserving metrics only
-

6. Commerce & Live Shop Subsystem

Purpose

Enable event-bound economic activity without turning events into malls.

Scope

- Event-scoped storefronts
- Vendor onboarding
- Revenue splits

Functional Requirements

- Events may host one or more shops
- Shops may sell:
 - Physical goods
 - Digital goods
 - Hybrid artifacts
- Shop availability must be time-bounded
- Revenue must be split according to event contracts
- Purchases must optionally require attendance

Non-Goals

- No global marketplace
- No infinite product catalogs

Technical Notes

- Commerce service isolated
 - Payment abstraction (Stripe + blockchain)
 - Inventory ephemeral by design
-

7. Spatial Map & Discovery Subsystem

Purpose

Provide geographic awareness without exposing sensitive locations.

Scope

- World map
- Event discovery
- Location obfuscation

Functional Requirements

- Map must support:
 - Global view
 - City clustering
 - Temporal filtering
- Private events must support:
 - Approximate locations
 - Delayed reveal
- Discovery must respect tier permissions

Non-Goals

- No real-time crowd tracking
- No exact coordinates for underground events

Technical Notes

- Mapbox / Cesium compatible
 - Layered visibility rules
 - Caching required for performance
-

8. Social Interaction Subsystem

Purpose

Enable connection without creating a dopamine feed.

Scope

- Event-bound interaction
- Temporary social graphs
- Post-event memory

Functional Requirements

- Social interactions must be event-anchored
- Temporary groups may form per event
- Post-event communication must decay or archive
- No persistent follower mechanics

Non-Goals

- No infinite scroll feeds
- No algorithmic content ranking

Technical Notes

- Graph DB optional
 - Time-to-live on interactions
 - Privacy first
-

9. Media & Memory Subsystem

Purpose

Capture cultural residue without exploiting creators.

Scope

- Media uploads
- Licensing
- Memory artifacts

Functional Requirements

- Media must be linked to:
 - Event
 - Creator
- Licensing scopes:
 - Event-only
 - Time-limited
 - Platform-wide (opt-in)
- Media access may require attendance proof

Non-Goals

- No public viral distribution
- No default platform ownership

Technical Notes

- Object storage
 - Metadata-rich indexing
 - Rights enforcement layer
-

10. Reputation & Trust Subsystem

Purpose

Measure contribution and reliability without social clout.

Scope

- Attendance history
- Ritual compliance
- Contribution scoring

Functional Requirements

- Reputation must be multi-dimensional
- Scores must decay over time
- Scores must be explainable
- Reputation must influence access but not visibility

Non-Goals

- No public leaderboards
- No popularity contests

Technical Notes

- Vector-based scoring possible
 - Explainability required
 - Stored separately from identity core
-

11. AI & Intelligence Layer

Purpose

Assist curation, moderation, and quality—not replace humans.

Scope

- Event recommendations
- Behavioral anomaly detection
- Trust assistance

Functional Requirements

- AI must be advisory by default
- AI outputs must be explainable
- AI must never autonomously ban users
- Models must be swappable

Non-Goals

- No opaque scoring
- No engagement optimization

Technical Notes

- Agent-based architecture compatible with FSIS
 - Feature flags for rollout
 - Human-in-the-loop enforced
-

12. Governance & Moderation Subsystem

Purpose

Resolve disputes and enforce norms across tiers.

Scope

- Moderation workflows
- Appeals

- Authority overrides

Functional Requirements

- Tier-based moderation powers
- Event-local moderators
- Escalation paths
- Audit trails for all actions

Non-Goals

- No centralized censorship feed
- No opaque moderation

Technical Notes

- Moderation as workflow engine
 - Full logging
 - DAO-style expansion possible
-

13. Platform Infrastructure

Purpose

Enable scalable, modular growth.

Scope

- APIs
- Services
- Deployment

Functional Requirements

- Modular microservices
- Event-driven architecture
- Real-time messaging
- Offline-capable critical paths

Non-Goals

- No monolith lock-in
- No chain-specific dependency

Technical Notes

- Vue + Quasar frontend
 - Python backend initially
 - Hybrid .NET integration
 - gRPC + REST
 - Containerized deployment
-

Final Note

This PRD intentionally **does not overspecify UI aesthetics or chain selection**, because those are *implementation details*, not core truths.

WeUP's real invariant is this:

Events are the atomic unit.
Identity is contextual.
Presence is measurable.
Culture is programmable—but not extractable.

Next logical steps:

- Convert this PRD into **epics & sprint plans**
- Produce **system sequence diagrams**
- Generate **data models per subsystem**
- Create **investor-safe technical summary**

Each reveals a different projection of the same architecture.